

Resumen teórico — Sistemas Operativos

Capítulos 1 a 3 de las Notas del curso y capítulo 1 de Tannenbaum & Bos

Tomás Rodeghiero

Sistemas Operativos — Universidad Nacional de Río Cuarto

2026

Resumen

Este documento es un resumen teórico de los temas iniciales de la materia Sistemas Operativos de la UNRC. Se basa en los capítulos 1, 2 y 3 de las *Notas del curso* (Marcelo Arroyo) y en el capítulo 1 del libro *Modern Operating Systems* (Tannenbaum & Bos). El objetivo es servir como material de estudio: las definiciones formales se acompañan de explicaciones intuitivas, ejemplos y conexiones entre conceptos, de modo que el lector pueda preparar tanto el examen teórico como el Práctico 1 con una comprensión sólida y articulada de los fundamentos.

Índice

1. Introducción	2
2. Estructura general de un sistema de cómputo	3
2.1. Capas de software y hardware	3
2.2. Hardware mínimo de interés	3
3. Historia y evolución de los sistemas operativos	4
4. Funciones y objetivos del sistema operativo	4
4.1. Objetivos generales	4
4.2. Multiplexión en tiempo y en espacio	5
4.3. Tipos de sistemas operativos	5
5. Kernel, modo usuario y modo kernel	5
5.1. El kernel	5
5.2. Modos de operación de la CPU	5
5.3. Mecanismos de protección	6
6. Arquitecturas de kernel	6
7. Booting: arranque del sistema	6
8. API, ABI y POSIX	7
9. Programas, procesos y espacios de direcciones	7
9.1. Programa vs. proceso	7
9.2. Imagen de memoria de un proceso	7
9.3. Process control block (PCB)	7
9.4. Espacio de direcciones y memoria virtual	8
9.5. Árbol de procesos	8
10. Llamadas al sistema (syscalls)	8
10.1. Definición y mecanismo	8

10.2. Las syscalls POSIX más usadas	8
11. Interrupciones y excepciones	9
11.1. Métodos de E/S	10
12. Multiprogramación, multitarea y tiempo compartido	10
13. El shell y la sesión de usuario	10
13.1. Sesiones, login y multiusuario	10
13.2. El shell	11
13.3. Operadores del shell	11
13.4. Variables de ambiente y PATH	11
14. Archivos y sistemas de archivos	11
14.1. Archivos como secuencia de bytes	11
14.2. Tipos de archivos en UNIX	12
14.3. Descriptores de archivo	12
14.4. Filesystem jerárquico	12
14.5. Permisos UNIX	12
15. Pipes y FIFOs (IPC)	13
15.1. Pipes sin nombre	13
15.2. FIFOs (pipes con nombre)	13
16. Señales y alarmas	13
16.1. Señales	13
16.2. Alarmas	14
17. Hilos y concurrencia (introducción)	14
18. El lenguaje C y la cadena de herramientas	14
18.1. Tipos y construcciones	14
18.2. Pasos de compilación	14
18.3. Bibliotecas estáticas y dinámicas	15
18.4. make y los Makefiles	15
19. Resumen integrador	15
Referencias	15

1. Introducción

Una computadora moderna es un conjunto enorme de componentes heterogéneos: uno o varios procesadores, memoria principal, discos, periféricos de entrada y salida, controladores, buses, redes. Si cada programador tuviera que tratar directamente con todos esos detalles, el desarrollo de aplicaciones sería virtualmente imposible. El **sistema operativo** (SO) resuelve ese problema: es la capa de software que se interpone entre el hardware y los programas de usuario, les ofrece abstracciones uniformes y administra el uso de los recursos disponibles.

Definición

Un **sistema operativo** es la pieza de software que abstrae el hardware, administra los recursos del sistema (CPU, memoria, dispositivos, archivos) y ofrece a las aplicaciones servicios uniformes mediante *llamadas al sistema*.

Las dos formas habituales de mirar al SO son complementarias:

- Como **máquina extendida**: oculta la fealdad del hardware (registros del controlador de disco, bits de estado de cada periférico, particularidades de cada arquitectura) y presenta a los programas un modelo limpio basado en archivos, procesos, sockets, etc. Tannenbaum lo resume diciendo que “el SO convierte lo feo en lo bello”.
- Como **administrador de recursos**: la CPU, la memoria y los dispositivos son recursos limitados que múltiples procesos quieren usar al mismo tiempo. El SO multiplexa el acceso de forma ordenada, justa y protegida.

Idea intuitiva

Si pensamos en un editor de video, un navegador y un cliente de correo abiertos a la vez en una notebook, ninguno de los tres se entera del otro. El SO logra esa ilusión multiplexando el procesador, partiendo la memoria en zonas privadas y arbitrando el acceso al disco y a la red.

2. Estructura general de un sistema de cómputo

2.1. Capas de software y hardware

Las Notas del curso presentan una estratificación típica:

1. **Hardware**: CPUs, RAM, dispositivos.
2. **Kernel del SO**: corre en *modo kernel*, tiene acceso total al hardware.
3. **Bibliotecas y utilidades del sistema**: `libc`, shells, compiladores, servicios (*daemons*). Corren en *modo usuario* pero son piezas del entorno operativo.
4. **Aplicaciones de usuario**: programas que interactúan con el SO mediante la API que provee `libc` y otras bibliotecas.

La frontera entre el modo usuario y el modo kernel es central para la seguridad y la estabilidad del sistema. Solo el código del kernel puede ejecutar instrucciones privilegiadas como deshabilitar interrupciones, acceder a puertos de dispositivos o manipular las tablas de páginas.

2.2. Hardware mínimo de interés

- **CPU**. Ejecuta el ciclo *fetch-decode-execute*. Posee registros generales, registros especiales (program counter, stack pointer, *Program Status Word*), un conjunto de instrucciones propio (x86, ARM, RISC-V, ...) y *modos de operación*. Las CPUs modernas usan *pipelines* y diseños *superscalares* que ejecutan varias instrucciones en paralelo, e incluso son *multi-core*.
- **ROM**. Memoria no volátil con el firmware de arranque (BIOS o UEFI).
- **RAM**. Memoria principal volátil donde reside lo que la CPU está ejecutando.
- **Buses y puertos de E/S**. Conectan la CPU con dispositivos. Los *controladores* (controllers) ofrecen registros que el kernel programa para iniciar transferencias.
- **DMA**. Transfiere datos entre dispositivo y memoria sin intervención de la CPU; al terminar, dispara una interrupción.

- **Jerarquía de memoria.** Registros, caché L1/L2, RAM, disco, todo con capacidades crecientes y velocidades decrecientes.

Idea intuitiva

La idea fundamental para entender lo que viene es: la CPU pasa la mayor parte del tiempo *esperando*. Sin algún mecanismo que aproveche el tiempo libre durante una operación de E/S, la máquina sería un desperdicio. El SO se construye alrededor de esa observación.

3. Historia y evolución de los sistemas operativos

Conviene mirar la evolución para entender por qué los conceptos modernos existen.

1. **Primera generación (1945–1955), tubos de vacío.** No había SO. Los programas eran *auto-contenidos*: cada uno incluía su propio código de E/S y rutinas matemáticas. Se cargaban manualmente vía *plugboards* o cintas.
2. **Segunda generación (1955–1965), transistores y batch systems.** Aparecen los *lotes* (*batch*): una pila de *jobs* se ejecuta secuencialmente. El SO incipiente gestiona la sucesión de comandos (\$JOB, \$FORTRAN, \$LOAD, \$RUN, \$END).
3. **Tercera generación (1965–1980), circuitos integrados y multiprogramación.** La CPU se vuelve más rápida que la E/S, así que se carga *más de un programa* en memoria a la vez y, al bloquearse uno por E/S, otro toma la CPU. Esta es la idea de **multiprogramación**. El *spooling* permite leer nuevos jobs desde disco. Aparece el **tiempo compartido** (**timesharing**) para dar respuesta interactiva (CTSS, MULTICS). Surge UNIX.
4. **Cuarta generación (1980–presente), computadoras personales.** CP/M, MS-DOS, MacOS, Windows, Linux. Los SO se vuelven *multiusuario* y *multitarea* en hardware protegido. Se popularizan las interfaces gráficas (GUI).
5. **Quinta generación (1990–presente), móviles.** Symbian, iOS, Android. Misma teoría, pero adaptada a recursos limitados, consumo y entornos táctiles.

Una idea recurrente que aparece en el libro de Tannenbaum: las tecnologías y técnicas se reinventan cíclicamente (“ontogeny recapitulates phylogeny”). Cada nueva clase de dispositivos (minicomputadoras, micros, smartphones, smartcards) suele empezar *primitiva* y va incorporando, a medida que el hardware lo permite, las mismas técnicas que ya existían en mainframes.

4. Funciones y objetivos del sistema operativo

4.1. Objetivos generales

- **Abstraer el hardware.** Por ejemplo, en UNIX los dispositivos aparecen como *archivos especiales* en `/dev`, y el almacenamiento masivo se modela como un *sistema de archivos* con directorios y nombres simbólicos.
- **Administrar recursos.** Multiplexa la CPU entre procesos, particiona la memoria, arbitra el acceso a los dispositivos.
- **Proveer servicios.** Las *llamadas al sistema* ofrecen E/S, manejo de procesos, archivos, comunicación, redes, sincronización.
- **Proteger el sistema.** Aislar los procesos entre sí y proteger al kernel de errores y abusos del software de usuario.

4.2. Multiplexión en tiempo y en espacio

Tannenbaum distingue dos modos de compartir un recurso:

- **Multiplexión en tiempo:** los usuarios o procesos se turnan. La CPU es un caso paradigmático: en un instante dado solo puede ejecutar un proceso, así que el SO conmuta entre ellos rápidamente. Otro ejemplo es la cola de impresión.
- **Multiplexión en espacio:** cada cliente recibe una *porción* del recurso al mismo tiempo. La memoria principal se divide entre procesos y el espacio de disco entre archivos.

4.3. Tipos de sistemas operativos

Las Notas y el libro coinciden en un *zoo* de variantes:

- **Mainframes:** priorizan el procesamiento de muchos jobs en paralelo (batch, transaccional, timesharing).
- **Servidores:** multiusuario, atienden clientes por red.
- **Multiprocesadores:** con varios CPUs (o cores) en una sola máquina.
- **PCs y portátiles (escritorio):** buscan dar buen soporte a un usuario interactivo.
- **Móviles:** multiplican GUIs táctiles, gestión agresiva de la batería y servicios de red.
- **Embedded y RTOS:** corren en dispositivos donde el software es fijo (microondas, automóviles, control industrial). Los *RTOS* garantizan respuestas dentro de plazos acotados; pueden ser *hard* (deadlines absolutos) o *soft*.
- **Sensor-node y smartcards:** extremadamente reducidos.

5. Kernel, modo usuario y modo kernel

5.1. El kernel

Definición

El **kernel** es el núcleo del sistema operativo: la porción de código que corre en *modo supervisor* y tiene acceso completo al hardware. Incluye el manejo de procesos, la gestión de memoria, los *device drivers*, el sistema de archivos y la pila de red.

Un programa de usuario común *no* es parte del kernel, aun si es fundamental (la *libc*, el shell, los servicios de impresión viven en *user space*). La frontera precisa puede ser borrosa: en sistemas tipo microkernel hasta el sistema de archivos puede vivir en modo usuario.

5.2. Modos de operación de la CPU

Las CPUs modernas implementan al menos dos *niveles de privilegio*:

- **Modo kernel (supervisor).** Toda instrucción es legal: habilitar/deshabilitar interrupciones, acceder a puertos, modificar registros de control de memoria, cambiar el modo.
- **Modo usuario.** Solo un subconjunto seguro de instrucciones está permitido. Cualquier intento de ejecutar una instrucción privilegiada genera una excepción y el kernel toma el control.

El bit que determina el modo está en el *Program Status Word* (PSW). El cambio de modo solo se produce mediante mecanismos controlados: una interrupción, una excepción o una llamada al sistema.

5.3. Mecanismos de protección

Para que la multiprogramación sea segura se necesitan, según las Notas:

- **Confinamiento:** cada proceso solo puede acceder a su propio espacio de memoria. La CPU genera una *segmentation fault* si lo viola.
- **Control:** los procesos no deben poder ejecutar instrucciones privilegiadas que les permitan tomar el control del kernel.

Idea intuitiva

Sin protección, un *bug* en un programa podría apagar el sistema o robar datos de otro. El hardware moderno (MMU, anillos de privilegio) está diseñado precisamente para ofrecerle al SO los mecanismos necesarios para implementar esos invariantes.

6. Arquitecturas de kernel

Tanto las Notas como el libro discuten varios estilos de diseño:

- **Monolíticos.** Todos los subsistemas se compilan y enlazan en un único ejecutable; se invocan entre sí como funciones ordinarias. Son simples y eficientes, pero un bug en un módulo puede afectar a todo el kernel. Ejemplos: UNIX clásico, Linux, FreeBSD.
- **Microkernels.** El kernel es muy pequeño y solo provee mecanismos básicos (planificación, IPC). Los subsistemas (filesystems, drivers, redes) corren como procesos independientes que se comunican por mensajes. Más robustos pero con sobrecarga de IPC. Ejemplos: Minix, L4, MS-Windows moderno (en parte).
- **Híbridos.** Combinan ambas ideas; ejemplo: macOS, que sobre el microkernel Mach corre buena parte de BSD UNIX como servicios.
- **Máquinas virtuales.** Un *hypervisor* multiplexa el hardware entre varios SO huéspedes, cada uno corriendo encima de su propio entorno virtual. Ejemplos: KVM, Xen, VMware.
- **Contenedores.** *No son virtualización:* comparten un mismo kernel, pero aíslan grupos de procesos y recursos (filesystems, redes virtuales, namespaces). En GNU-Linux se implementan con *cgroups*; ejemplo de uso: Docker.

Un principio de diseño recurrente: separar *mecanismos* (en las capas bajas) de *políticas* (en las capas altas) para favorecer la flexibilidad. Por ejemplo, el kernel implementa el mecanismo de cambio de contexto y el planificador define la política.

7. Booting: arranque del sistema

Resumen del proceso descrito en las Notas:

1. Al encender, la CPU comienza a ejecutar código en una dirección fija de la ROM: el *firmware* (BIOS o UEFI).
2. El firmware verifica RAM, dispositivos, y carga el primer sector del disco de arranque (MBR) o, en UEFI, un ejecutable de la partición EFI.
3. Ese código suele ser un *bootloader* (GRUB) que carga el kernel del SO en memoria y le transfiere el control.
4. El kernel inicializa subsistemas (drivers, planificador, sistemas de archivos) y, finalmente, lanza el primer proceso de usuario (en UNIX: *init*, *systemd* o *launchd*). Ese proceso es el ancestro de todos los demás.

8. API, ABI y POSIX

Definición

La **API** (*Application Programming Interface*) es la interfaz a nivel de código fuente: declaraciones de funciones, tipos de datos y constantes que el programador usa para invocar servicios del sistema.

Definición

La **ABI** (*Application Binary Interface*) es la interfaz de bajo nivel que especifica cómo se pasan argumentos, cómo se llaman las funciones, qué registros se preservan, y qué formato tienen los ejecutables (ELF en GNU-Linux, Mach-O en macOS, PE en Windows).

Definición

POSIX (*Portable Operating System Interface*) es un estándar del IEEE que define una API mínima común para los sistemas tipo UNIX. Especifica syscalls (`open`, `read`, `write`, `fork`, `exec`, ...), utilidades de shell y semánticas asociadas. Un programa que respeta POSIX corre, con mínimas adaptaciones, en cualquier UNIX certificado.

9. Programas, procesos y espacios de direcciones

9.1. Programa vs. proceso

Un **programa** es un archivo ejecutable: una secuencia de instrucciones de máquina y datos almacenados en disco. Un **proceso**, en cambio, es ese programa *en ejecución*, con todo el contexto que necesita: registros, memoria, archivos abiertos, etc.

Definición

Un **proceso** es una instancia de un programa en ejecución. En un sistema multitarea pueden coexistir múltiples instancias del mismo programa.

9.2. Imagen de memoria de un proceso

En UNIX un proceso tiene tres áreas principales (segmentos):

- **Texto (text)**: las instrucciones del programa. Es de solo lectura y se puede compartir entre instancias.
- **Datos (data)**: variables globales y constantes inicializadas o no. Crece con `brk/sbrk` (y, en la práctica, vía `malloc`).
- **Pila (stack)**: marcos de activación de funciones, con parámetros, variables locales y direcciones de retorno. Crece en sentido contrario a los datos.

Entre la pila y los datos queda un hueco de direcciones libres. La pila crece hacia abajo (direcciones decrecientes) y los datos hacia arriba.

9.3. Process control block (PCB)

El kernel representa cada proceso con una estructura llamada *PCB* (o *task descriptor*) que contiene:

- Identificador del proceso (`pid`).
- Estado: `RUNNING`, `READY`, `WAITING`, `ZOMBIE`, ...

- Valor de los registros, incluido el *program counter* y el *stack pointer*.
- Lista de áreas de memoria en uso.
- Tabla de archivos abiertos (descriptores).
- Información de seguridad (UID, GID), señales pendientes, prioridad, padre, hijos.

9.4. Espacio de direcciones y memoria virtual

Cada proceso ve su propio espacio de direcciones lineal, típicamente desde la dirección 0 hasta un máximo (por ejemplo, $2^{64} - 1$ en una plataforma de 64 bits). La *MMU* (*Memory Management Unit*) traduce esas direcciones *virtuales* a direcciones *físicas* de RAM, posibilitando además la **memoria virtual**: páginas que no se usan se llevan al disco y se traen a demanda. Esto permite ejecutar programas más grandes que la memoria física.

9.5. Árbol de procesos

Cuando un proceso crea hijos, los hijos pueden a su vez crear nietos, formando un árbol. Ejemplo típico: `init` es la raíz, lanza `getty/login`, que lanzan el shell del usuario, que a su vez lanza los comandos.

10. Llamadas al sistema (syscalls)

10.1. Definición y mecanismo

Definición

Una **llamada al sistema** es una función que un proceso de usuario invoca para pedir un servicio al kernel. Se implementan con una instrucción especial (*trap*, *syscall*, *software interrupt*) que provoca un cambio de modo.

El proceso típico de una syscall, según la figura 1-17 del libro:

1. El programa apila los parámetros en la pila o los pone en registros, según la ABI.
2. Llama a la función de biblioteca correspondiente (por ejemplo, `read` de `libc`).
3. Esa función coloca el *número de syscall* en un registro convenido.
4. Ejecuta la instrucción `TRAP` (o `syscall`, en x86-64).
5. La CPU cambia a modo kernel y salta a un punto fijo del kernel: el *trap handler*.
6. El handler usa el número para indexar una tabla de despachadores y llama a la rutina del kernel que corresponde.
7. Cuando la rutina termina, el kernel restaura el contexto y retorna al punto siguiente al `TRAP`, ya en modo usuario.

Idea intuitiva

Una syscall es *como* una llamada a función ordinaria, pero con dos diferencias importantes: (i) cambia de modo de privilegio y (ii) no salta a una dirección arbitraria, sino a un punto fijo del kernel.

10.2. Las syscalls POSIX más usadas

A continuación, una selección de las que aparecen en las Notas y en el libro (Figura 1-18 de Tannenbaum), agrupadas por área:

Syscall	Función
<i>Procesos</i>	
<code>pid = fork()</code>	Crea un proceso hijo, copia exacta del padre.
<code>exit(status)</code>	Finaliza el proceso con el código indicado.
<code>waitpid(pid,&st,opts)</code>	Espera la terminación de un hijo.
<code>execve(path,argv,envv)</code>	Reemplaza la imagen de memoria por otro programa.
<code>getpid() / getppid()</code>	Devuelve el PID propio o del padre.
<code>kill(pid, sig)</code>	Envía una señal a un proceso.
<code>sleep(n)</code>	Bloquea al proceso <i>n</i> segundos.
<i>Archivos</i>	
<code>fd = open(path,mode)</code>	Abre un archivo y devuelve un descriptor.
<code>n = read(fd,buf,n)</code>	Lee <i>n</i> bytes en el buffer.
<code>n = write(fd,buf,n)</code>	Escribe <i>n</i> bytes desde el buffer.
<code>close(fd)</code>	Cierra el descriptor.
<code>lseek(fd,off,whence)</code>	Mueve el offset interno de E/S.
<code>stat(path,&st)</code>	Obtiene metadatos del archivo.
<code>dup(fd)</code>	Duplica un descriptor en el siguiente slot libre.
<code>pipe(p)</code>	Crea un pipe sin nombre.
<i>Directorios y filesystems</i>	
<code>chdir(dir)</code>	Cambia el directorio de trabajo.
<code>mkdir(name,mode)</code>	Crea un directorio.
<code>link(f1,f2)</code>	Crea un <i>hard link</i> .
<code>unlink(name)</code>	Elimina una entrada de directorio.
<code>mount/umount</code>	Monta o desmonta un filesystem.
<i>Misceláneas</i>	
<code>chmod(name,mode)</code>	Cambia los bits de protección.
<code>time(&t)</code>	Devuelve segundos desde el 1/1/1970.
<code>alarm(n)</code>	Programa la señal SIGALRM.

Cuadro 1: Algunas de las syscalls POSIX más comunes.

11. Interrupciones y excepciones

Las interrupciones son la forma estándar en que el hardware notifica eventos asincrónicos al software:

- Un dispositivo termina una transferencia y dispara una *interrupt request* (IRQ).
- El controlador de interrupciones le avisa a la CPU.
- La CPU termina la instrucción actual, salva el PC y el PSW en la pila, conmuta a modo kernel y salta al *interrupt handler* indexado en el *interrupt vector*.
- El handler atiende la interrupción y retorna; la CPU restaura el contexto y continúa el flujo previo.

Las *excepciones* son disparadas por la propia CPU en respuesta a condiciones del programa: división por cero, acceso a memoria prohibida, instrucción ilegal. El kernel decide si terminar el proceso, ignorar el problema o convertirlo en una señal manejable.

Un detalle importante: las syscalls también se implementan con un mecanismo similar al de las interrupciones (instrucción TRAP o *software interrupt*), por lo que comparten infraestructura.

11.1. Métodos de E/S

Tannenbaum distingue tres formas de hacer entrada/salida:

- **Busy waiting (polling):** el driver inicia la operación y se queda en un bucle preguntando por el estado del dispositivo. Simple, pero desperdicia CPU.
- **Por interrupciones:** el driver inicia la operación y retorna; cuando el dispositivo termina, dispara una interrupción que despierta al proceso.
- **DMA:** el chip DMA copia datos entre dispositivo y memoria sin intervenir la CPU; al terminar, una interrupción cierra el ciclo.

12. Multiprogramación, multitarea y tiempo compartido

Definición

La **multiprogramación** es la técnica de mantener varios procesos cargados en memoria al mismo tiempo y multiplexar la CPU entre ellos para mantenerla ocupada incluso durante operaciones de E/S.

Definición

La **multitarea** es la propiedad de un SO de ejecutar aparentemente varios procesos en simultáneo. Puede ser *cooperativa* (los procesos ceden voluntariamente la CPU) o *preemptiva* (el kernel les puede sacar la CPU).

Definición

El **tiempo compartido (timesharing)** es una variante de multiprogramación interactiva en la que el kernel, ante una interrupción periódica del reloj, puede quitarle la CPU al proceso corriente y dársela a otro.

El cambio de proceso se llama **context switch**. Implica salvar el estado del proceso saliente (registros, PC, PSW, mappings de la MMU) en su PCB y restaurar el del proceso entrante. Es una operación relativamente costosa que el SO trata de minimizar.

Idea intuitiva

La sensación de simultaneidad surge porque el SO multiplexa muy rápido. En un único instante, una CPU mononúcleo solo está ejecutando *un* proceso; la ilusión de paralelismo se construye sobre el hecho de que cada proceso recibe muchas rebanadas pequeñas de tiempo por segundo.

13. El shell y la sesión de usuario

13.1. Sesiones, login y multiusuario

Un sistema multiusuario maneja *sesiones*. Una terminal autentica al usuario (*login*), levanta un shell con sus credenciales (UID, GID, HOME) y queda lista para recibir comandos. El cierre de la sesión (Ctrl-D o *exit*) termina ese shell; *init* reinicia el ciclo en esa terminal.

13.2. El shell

Definición

Un **shell** es un intérprete REPL (read-eval-print loop) que recibe comandos del usuario, los analiza y ejecuta el programa correspondiente. Implementa además operadores que combinan procesos (redirección, secuencia, pipes, condicionales).

Cada vez que el usuario escribe un comando, el shell hace **fork**, en el hijo aplica las redirecciones pedidas y ejecuta **execve**; el padre espera con **waitpid**. El esqueleto mínimo aparece en Tannenbaum (Figura 1-19) como:

```
1 while (TRUE) {
2     type_prompt();
3     read_command(command, parameters);
4     if (fork() != 0) {
5         waitpid(-1, &status, 0); /* padre espera al hijo */
6     } else {
7         execve(command, parameters, 0); /* hijo ejecuta el comando */
8     }
9 }
```

13.3. Operadores del shell

- **Secuenciales:** `cmd1 ; cmd2` ejecuta los dos en orden, sin importar el éxito.
- **Condicionales:** `cmd1 && cmd2` ejecuta `cmd2` solo si `cmd1` terminó con `exit_code = 0`; `cmd1 || cmd2` hace lo contrario.
- **Background:** `cmd &` lanza el comando como proceso de fondo; el shell asigna un *job id* y devuelve el control de inmediato.
- **Redirección:** `cmd > file` (sobreescribe), `cmd » file` (append), `cmd < file`, `cmd 2> file` (redirige stderr).
- **Pipes:** `cmd1 | cmd2` envía la salida estándar del primero a la entrada estándar del segundo.

13.4. Variables de ambiente y PATH

Cada sesión tiene un conjunto de *variables de ambiente* que se heredan a los procesos hijos en el tercer parámetro de `main` (`char *envv[]`). Las relevantes:

- **HOME:** directorio personal.
- **PWD:** directorio actual.
- **USER:** usuario actual.
- **SHELL:** intérprete configurado.
- **PATH:** lista de directorios separados por `:` donde el shell busca ejecutables. Si el comando se invoca con barras (`./prog` o `/usr/bin/prog`) se toma como ruta directa; si no, se busca en **PATH**.

14. Archivos y sistemas de archivos

14.1. Archivos como secuencia de bytes

Para la mayoría de los SO modernos, un archivo es simplemente una secuencia de bytes. La estructura interna (texto, ELF, JPEG, ...) la imponen las aplicaciones, no el kernel. Los archivos

se identifican por su *path*, que puede ser *absoluto* (empieza en la raíz /) o *relativo* al directorio de trabajo.

Cada archivo tiene metadatos (tamaño, propietario, permisos, timestamps, contador de enlaces, tipo) almacenados en una estructura llamada **inodo**. La entrada de directorio asocia un nombre con un inodo.

14.2. Tipos de archivos en UNIX

- **Regulares:** datos de usuario, código binario, texto.
- **Directorios:** contienen pares (*nombre*, *inodo*). Cada uno tiene siempre `.` y `...`
- **Archivos especiales** (en `/dev`): representan dispositivos. Pueden ser *block special* (discos) o *character special* (terminales, mouse).
- **Pipes con nombre (FIFO):** archivos especiales para comunicación entre procesos no emparentados.
- **Enlaces:** ya sean *symbolic* (apuntan por ruta) o *hard* (otra entrada al mismo inodo).

14.3. Descriptores de archivo

Definición

Un **descriptor de archivo** es un entero no negativo que el kernel le asigna a un proceso al abrir un recurso. El proceso lo usa en las syscalls posteriores (`read`, `write`, `close`, `lseek`).

Cada proceso, al ser lanzado por `init`, hereda tres descriptores conectados a la terminal:

- 0: *stdin* (entrada estándar, teclado).
- 1: *stdout* (salida estándar, pantalla).
- 2: *stderr* (salida de errores, pantalla).

La syscall `dup(fd)` duplica un descriptor en el primer slot libre; combinada con `close` y `open`, es la base de las redirecciones de E/S del shell.

14.4. Filesystem jerárquico

Las Notas y el libro coinciden en describir el filesystem como un árbol con un único directorio raíz /. Los filesystems removibles (CDs, USB, particiones) se incorporan al árbol mediante la syscall `mount`, que los *ata* a un directorio existente. `umount` hace lo contrario.

14.5. Permisos UNIX

Cada archivo tiene un *modo* de 9 bits agrupados en tres ternas (dueño, grupo, otros) con los flags **r** (lectura), **w** (escritura) y **x** (ejecución). Por ejemplo, el modo `rw-r--r--` permite al dueño leer, escribir y ejecutar; al grupo y a otros, solo leer. Para directorios, **x** significa permiso para *atravesarlos*.

15. Pipes y FIFOs (IPC)

15.1. Pipes sin nombre

Definición

Un **pipe** es un buffer acotado en el kernel con dos extremos (lectura y escritura) que implementa el patrón productor/consumidor. La syscall `pipe(p)` crea uno y devuelve dos descriptors: `p[0]` (lectura) y `p[1]` (escritura).

Los pipes solo permiten comunicar procesos *emparentados* porque los descriptors se comparten vía `fork`. El comando shell `cmd1 | cmd2` se implementa así:

```
1 int p[2];
2 pipe(p);
3 if (fork() == 0) { /* primer hijo: cmd1 */
4     close(p[0]); /* no lee del pipe */
5     dup2(p[1], STDOUT_FILENO); /* su stdout va al pipe */
6     close(p[1]);
7     execvp("cmd1", args1);
8 }
9 if (fork() == 0) { /* segundo hijo: cmd2 */
10    close(p[1]); /* no escribe en el pipe */
11    dup2(p[0], STDIN_FILENO); /* su stdin viene del pipe */
12    close(p[0]);
13    execvp("cmd2", args2);
14 }
15 close(p[0]); close(p[1]);
16 wait(NULL); wait(NULL);
```

Importante: si el padre no cierra ambos extremos, el lector nunca verá EOF y se quedará bloqueado para siempre.

15.2. FIFOs (pipes con nombre)

Un **FIFO** es un pipe que tiene un nombre en el sistema de archivos. Se crea con `mkfifo(path, mode)` y se abre con `open/read/write/close`. Permite comunicar procesos *independientes* (sin parentesco). Los datos no se almacenan en disco; el archivo solo da un punto de encuentro y los datos viven en el buffer del kernel mientras hay procesos conectados.

16. Señales y alarmas

16.1. Señales

Definición

Una **señal** es una notificación asíncrona de un evento enviada por el kernel (o por otro proceso vía `kill`) a un proceso destinatario. Es el “equivalente software” de una interrupción.

Algunos ejemplos:

- `SIGINT` (Ctrl-C): interrumpir el proceso.
- `SIGTSTP` (Ctrl-Z): suspenderlo.
- `SIGSEGV`: acceso a memoria inválida.
- `SIGFPE`: error aritmético (división por cero).
- `SIGALRM`: alarma vencida.

- **SIGKILL**: terminación incondicional. No se puede manejar ni ignorar.

Un proceso puede instalar un *handler* con **signal** o **sigaction**; cuando el kernel le entrega la señal, le *salta* al handler antes de continuar el flujo normal. Si no hay handler, el kernel ejecuta la acción por defecto (terminar, ignorar, suspender, según la señal).

Restricción importante: dentro de un handler solo deben invocarse funciones *async-signal-safe*: **read**, **write**, **_exit**, **sigaction**, etc. Las funciones de **stdio** (**printf**, **fgets**, **scanf**) usan buffers internos que pueden quedar en estado inconsistente al ser interrumpidos.

16.2. Alarmas

La syscall **alarm(n)** le pide al kernel que envíe **SIGALRM** luego de **n** segundos. Es la base para implementar *timeouts* sobre operaciones bloqueantes:

```
1 signal(SIGALRM, on_alarm);
2 alarm(5); /* 5 segundos */
3 nread = read(STDIN_FILENO, buf, n);
4 alarm(0); /* cancelar si llego dato */
```

17. Hilos y concurrencia (introducción)

Un proceso clásico tiene un único hilo de ejecución (un *program counter*, una pila). Un **thread** es un hilo de ejecución independiente que comparte el espacio de direcciones con otros hilos del mismo proceso. Los threads se introducen en el capítulo 1 como “procesos livianos” y permiten paralelizar dentro de una misma aplicación. La sincronización (mutexes, semáforos, condiciones) es necesaria para acceder a datos compartidos sin condiciones de carrera; ese tema se desarrolla en prácticos posteriores.

18. El lenguaje C y la cadena de herramientas

Las Notas dedican el capítulo 3 a presentar C y la *toolchain* porque son el vehículo natural para programar el SO y las aplicaciones que ejercitan sus syscalls. Conceptos relevantes:

18.1. Tipos y construcciones

- Tipos básicos: **char**, **int**, **long** (con/sin signo), **float**, **double**.
- Compuestos: arreglos, strings (terminados en **\0**), **struct**, **union**.
- Punteros y aritmética de punteros.
- Funciones: pasaje por valor, simulación de pasaje por referencia con punteros, funciones variádicas como **printf**.
- Módulos: separación en archivos **.c** y cabeceras **.h**; **#include** y declaraciones **extern**.

18.2. Pasos de compilación

1. **Pre-procesado** (**cpp**): expande **#include**, **#define** y demás directivas.
2. **Compilación** (**cc1**): traduce de C a *assembly*.
3. **Ensamblado** (**as**): genera archivos objeto **.o**.
4. **Linking** (**ld**): combina objetos y bibliotecas en un ejecutable, resolviendo símbolos y referencias externas.

18.3. Bibliotecas estáticas y dinámicas

- **Estáticas** (`libfoo.a`): el código de la biblioteca se incrusta en el ejecutable durante el linking. Resultado: ejecutables más grandes pero auto-contenidos.
- **Dinámicas** (`libfoo.so` en Linux, `.dylib` en macOS, `.dll` en Windows): el ejecutable solo guarda referencias; el *dynamic linker* las resuelve en tiempo de carga (o, con *lazy linking*, en la primera invocación). Ejecutables más chicos y biblioteca compartida en memoria entre procesos.

18.4. make y los Makefiles

`make` automatiza la compilación a partir de reglas **target: dependencias**. Recompila solo lo necesario comparando *timestamps*. Las *autotools* (`autoconf`, `automake`) generan Makefiles portables.

19. Resumen integrador

Para fijar las ideas, conviene visualizar cómo todos estos conceptos encajan al ejecutar un comando de shell, por ejemplo:

```
$ ls -l | wc -l > cuentas.txt
```

1. El usuario tipea el comando en la terminal. El *tty* driver del kernel envía los caracteres al shell.
2. El shell parsea la línea: detecta el pipe (`|`) y la redirección (`>`).
3. Llama a `pipe(p)` para crear un pipe entre los dos comandos.
4. Hace `fork` dos veces:
 - El primer hijo redirige su `stdout` al extremo de escritura del pipe (`dup2`) y ejecuta `ls -l` con `execve`.
 - El segundo hijo redirige su `stdin` al extremo de lectura, abre `cuentas.txt` con `O_WRONLY|O_CREAT|O_TRUNC`, redirige su `stdout` a ese archivo y ejecuta `wc -l`.
5. El padre cierra ambos extremos del pipe y llama a `waitpid` dos veces para esperar a los hijos.
6. Cada `execve` hace que el kernel cargue el ELF correspondiente, prepare el espacio de direcciones (texto, datos, pila), enlace dinámicamente `libc` y salte a `main`.
7. Los procesos ejecutan, hacen `read` y `write` contra los descriptores que el shell les preparó. Cuando `ls` termina, cierra sus descriptores y el extremo de escritura del pipe se libera; `wc` ve EOF, termina y cierra su archivo de salida.
8. El shell recoge los exit status, actualiza `$?` y vuelve a mostrar el prompt.

Detrás de un comando aparentemente simple operan, simultáneamente: *procesos, multiplexión de CPU, descriptores y redirección, pipes como IPC, permisos del filesystem, libc dinámica, cambios de modo* para cada syscall y *interrupciones* cada vez que el kernel hace context switch o atiende una E/S. El estudio de los sistemas operativos consiste, precisamente, en entender cómo todas esas piezas funcionan en conjunto.

Referencias

1. Marcelo Arroyo. *Notas del curso de Sistemas Operativos*, capítulos 1, 2 y 3. UNRC, 2026.
<https://marceloarroyo.gitlab.io/cursos/SO/>

2. Andrew S. Tanenbaum, Herbert Bos. *Modern Operating Systems*, capítulo 1, 5ª edición. Pearson, 2024.
3. Remzi y Andrea Arpaci-Dusseau. *Operating Systems: Three Easy Pieces*. <https://pages.cs.wisc.edu/~remzi/OSTEP/>
4. Avi Silberschatz, Peter B. Galvin, Greg Gagne. *Operating System Concepts*, 9ª edición.
5. POSIX.1-2017 / Single UNIX Specification. The Open Group.